

Тестовое задание: Notification Preferences Service

Стек: TypeScript, Node.js, PostgreSQL

Ориентир по времени: 6–8 часов (можно делать частями)

Результат: ссылка на репозиторий через **форму** + короткое README с инструкцией, как запустить сервис и тесты.

Контекст

У нас платформа, где разные продуктовые модули отправляют пользователям уведомления по нескольким каналам: email, SMS, мессенджеры, пуши. Нужно спроектировать и реализовать сервис управления предпочтениями уведомлений, который служит единым источником правды для остальных компонентов.

Сервис отвечает за то, какие типы уведомлений и по каким каналам можно отправлять конкретному пользователю с учётом его выбора, дефолтных настроек и глобальных политик.

Задача:

Реализовать **Notification Preferences Service** на TypeScript и Node.js с использованием PostgreSQL.

Сервис должен уметь:

1. Хранить настройки:

- дефолтные предпочтения для новых пользователей;
- индивидуальные настройки пользователя по типам уведомлений и каналам;
- глобальные политики (например, запрет определённого типа уведомлений по региону).

2. Предоставлять API для:

- получения текущих предпочтений пользователя;
- изменения предпочтений пользователя:
 - включение/выключение типа уведомления по каналу;
 - настройка quiet hours в заданной таймзоне;
- проверки возможности отправки уведомления:
 - вход: `userId`, `notificationType`, `channel`, `region`, `datetime`;
 - выход: решение `allow` / `deny` и краткое объяснение причины.

3. Учитывать бизнес-правила:

- дефолтные настройки;
- индивидуальные настройки пользователя;
- глобальные политики по типу/каналу/региону;
- quiet hours с учётом таймзоны;
- идемпотентность операций изменения настроек.

Поведение через сценарии:

Сервис должен вести себя так, чтобы можно было реализовать сценарии:

1. Новый пользователь и дефолты

- При создании нового пользователя у него есть набор дефолтных предпочтений.
- Пример: `transactional\_email` включён, `marketing\_email` выключен.

2. Изменение настроек пользователем

- Пользователь может отключить, например, маркетинговые email-уведомления.
- При запросе предпочтений это должно отражаться в ответе.
- При этом транзакционные email-уведомления остаются разрешёнными.

3. Quiet hours

- Пользователь задаёт quiet hours (например, с 22:00 до 08:00 в своей таймзоне).
- В это время маркетинговые пуши блокируются.
- Транзакционные уведомления могут оставаться разрешёнными.

4. Глобальные политики

- Может существовать глобальная политика, которая запрещает определённый тип уведомлений в конкретном регионе (например, `marketing\_sms` в регионе `EU`).
- При проверке возможности отправки уведомления такая политика должна быть учтена.

5. Идемпотентность

- Повторное применение одной и той же команды изменения настроек не ломает состояние.
- Пример: дважды подряд отключаем `marketing\_email` — результат должен быть таким, как будто отключили один раз.

Вы можете интерпретировать детали на своё усмотрение, если сохраняется здравый смысл и прозрачность.

Требования к реализации:

- TypeScript по всему backend-коду.
- Читаемая структура проекта: разделение доменной логики и инфраструктуры.
- Аккуратные доменные типы:
 - для `notificationType`, `channel`, `region` и т.п.;
 - работа с датами и таймзонами.

API:

Формат API на выбор: REST или GraphQL. Важно, чтобы его было легко вызвать и проверить.

Минимальный набор:

1. Получение предпочтений пользователя

Пример: `GET /users/:id/preferences`

2. Изменение предпочтений пользователя

Пример: `POST /users/:id/preferences`

Тело может содержать:

- включение/выключение типа уведомления по каналу;
- настройки quiet hours.

3. Проверка возможности отправки уведомления

Пример: `POST /evaluate`

Вход:

```
```json
{
 "userId": "user-1",
 "notificationType": "marketing_email",
 "channel": "email",
 "region": "EU",
 "datetime": "2026-05-21T21:30:00Z"
}
```
```

Выход:

```
```json
{
 "decision": "deny",
 "reason": "blocked_by_global_policy"
}
```
```

Форматы можно менять, если идея остаётся ясной.

Тесты:

Нужны тесты, которые покрывают ключевое поведение сервиса:

- дефолтные настройки для нового пользователя;
- изменение настроек пользователем;
- влияние quiet hours;
- влияние глобальных политик;
- идемпотентность.

Можно использовать любой знакомый вам тест-фреймворк.

Observability и эксплуатация:

- Базовое логирование ключевых событий:
 - изменения настроек;
 - решения `allow` / `deny` по отправке уведомлений.
- Если хотите, можете добавить структуру для метрик (счётчики, таймеры) или комментарии, где бы вы их внедрили.

Что прислать в итоге:

1. Ссылку на репозиторий (GitHub/GitLab/Bitbucket).
2. Короткое `README`, в котором описано:
 - как запустить проект (с учётом PostgreSQL);
 - как запустить тесты;
 - краткое описание архитектуры и основных решений;
 - что вы добавили бы дальше, если бы доводили этот сервис до продакшена.

Docker-окружение приветствуется, но не обязательно.

Главный фокус — на **качестве** архитектуры, типов, бизнес-логики и соответствии описанному поведению.